

# Object-Oriented Programming

## Unit 3: Inheritance in OO Design, Design Patterns & Iterator Pattern

MAKAUT B.Tech IT | Exam-Ready Notes with Java Code

### 1. What is Inheritance?

**Inheritance** is the OOP mechanism by which a **new class (subclass / child class)** **acquires the fields and methods of an existing class (superclass / parent class)** without re-writing them. It models the real-world **IS-A relationship**: a Dog IS-A Animal; a Car IS-A Vehicle.

The subclass can use the parent's members as-is, override them (change behaviour), hide them (for static members), or add entirely new members. The key Java keyword is **extends**.

#### Key terminology:

| Term       | Also known as                                | Meaning                                                    |
|------------|----------------------------------------------|------------------------------------------------------------|
| Superclass | Parent class / Base class                    | The class being inherited from                             |
| Subclass   | Child class / Derived class / Extended class | The class that inherits                                    |
| extends    | —                                            | Java keyword that establishes the inheritance relationship |
| super      | —                                            | Keyword to call the parent class's constructor or methods  |

**Important — what is and isn't inherited:** A subclass inherits all **public** and **protected** members (fields + methods + nested classes). It does **NOT** inherit **private** members (but can access them through public/protected getters). **Constructors are never inherited** — they must be explicitly invoked with `super(...)`.

#### 1.1 Basic Inheritance Example

```
// Superclass
class Animal {
    String name;
    int age;

    Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void eat() {
        System.out.println(name + " is eating.");
    }

    void breathe() {
        System.out.println(name + " breathes air.");
    }
}

// Subclass – inherits name, age, eat(), breathe()
class Dog extends Animal {
    String breed;
}
```

```

    Dog(String name, int age, String breed) {
        super(name, age);    // calls Animal's constructor – mandatory if parent has no default
    }

    this.breed = breed;

    // NEW method – specific to Dog
    void bark() {
        System.out.println(name + " says: Woof!");
    }

    // OVERRIDING eat() – changes inherited behaviour
    @Override
    void eat() {
        System.out.println(name + " is eating dog food.");
    }
}

class Main {
    public static void main(String[] args) {
        Dog d = new Dog("Rex", 3, "Labrador");
        d.eat();        // Dog's version – overridden
        d.breathe();    // Animal's version – inherited as-is
        d.bark();        // Dog's own method

        // IS-A check
        System.out.println(d instanceof Animal); // true
        System.out.println(d instanceof Dog);    // true
    }
}

```

## 2. Types of Inheritance in Java

Java supports 5 inheritance types, but only 4 are directly supported with classes. Multiple inheritance of **state** (extending two classes simultaneously) is intentionally prohibited to prevent the Diamond Problem — it is achieved through **interfaces** instead.

| Type         | Structure                                       | Java support         |
|--------------|-------------------------------------------------|----------------------|
| Single       | One subclass extends one superclass             | ✓ with class         |
| Multilevel   | A extends B; C extends A — chain of inheritance | ✓ with class         |
| Hierarchical | Multiple subclasses extend the same superclass  | ✓ with class         |
| Multiple     | One subclass extends two or more superclasses   | ✓ via interface only |
| Hybrid       | Combination of two or more of the above types   | ✓ via interface only |

### 2.1 Single Inheritance

```
class Vehicle {
    String brand;
    Vehicle(String brand) { this.brand = brand; }
    void start() { System.out.println(brand + " started."); }
}

class Car extends Vehicle {
    int doors;
    Car(String brand, int doors) {
        super(brand);
        this.doors = doors;
    }
    void honk() { System.out.println(brand + " beeps!"); }
}

class Main {
    public static void main(String[] args) {
        Car c = new Car("Toyota", 4);
        c.start(); // inherited from Vehicle
        c.honk();  // Car's own method
    }
}
```

### 2.2 Multilevel Inheritance

Chain: A → B → C. Constructor calls go top-down automatically.

```
class Vehicle {
    Vehicle() { System.out.println("Vehicle created"); }
    void fuelType() { System.out.println("Runs on fuel"); }
}

class FourWheeler extends Vehicle {
    FourWheeler() { System.out.println("FourWheeler created"); }
    void wheelCount() { System.out.println("Has 4 wheels"); }
}

class Car extends FourWheeler {
    Car() { System.out.println("Car created"); }
    void type() { System.out.println("This is a Car"); }
}
```

```

class Main {
    public static void main(String[] args) {
        Car c = new Car();
        // Output:
        // Vehicle created
        // FourWheeler created
        // Car created

        c.fuelType(); // from Vehicle (2 levels up)
        c.wheelCount(); // from FourWheeler (1 level up)
        c.type(); // Car's own
    }
}

```

## 2.3 Hierarchical Inheritance

One parent, multiple independent children. Most common in real codebases.

```

class Shape {
    String color;
    Shape(String color) { this.color = color; }
    double area() { return 0; } // to be overridden
    void display() {
        System.out.printf("%s area = %.2f\n", color + " " + getClass().getSimpleName(), area());
    }
}

class Circle extends Shape {
    double radius;
    Circle(String color, double radius) { super(color); this.radius = radius; }
    @Override double area() { return Math.PI * radius * radius; }
}

class Rectangle extends Shape {
    double w, h;
    Rectangle(String color, double w, double h) { super(color); this.w=w; this.h=h; }
    @Override double area() { return w * h; }
}

class Triangle extends Shape {
    double base, height;
    Triangle(String color, double b, double h) { super(color); this.base=b; this.height=h; }
    @Override double area() { return 0.5 * base * height; }
}

class Main {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle("Red", 5),
            new Rectangle("Blue", 4, 6),
            new Triangle("Green", 3, 8)
        };
        for (Shape s : shapes) s.display();
        // Red Circle area = 78.54
        // Blue Rectangle area = 24.00
        // Green Triangle area = 12.00
    }
}

```

## 2.4 Multiple Inheritance (via Interfaces)

Java blocks multiple class inheritance to avoid the **Diamond Problem** (ambiguity when two parents define the same method). Interfaces solve this — a class can **implement** as many interfaces as needed.

```
interface Flyable {
    default void fly() { System.out.println("Flying..."); }
    void maxAltitude(); // abstract – must be implemented
}

interface Swimmable {
    default void swim() { System.out.println("Swimming..."); }
    void maxDepth();
}

// Duck implements both interfaces – multiple inheritance of TYPE
class Duck implements Flyable, Swimmable {
    @Override public void maxAltitude() { System.out.println("Max altitude: 300m"); }
    @Override public void maxDepth()    { System.out.println("Max depth: 2m"); }
}

class Main {
    public static void main(String[] args) {
        Duck d = new Duck();
        d.fly();           // from Flyable
        d.swim();          // from Swimmable
        d.maxAltitude();
        d.maxDepth();
    }
}
```

## 2.5 Hybrid Inheritance

```
class Vehicle {
    void vehicleType() { System.out.println("I am a vehicle"); }
}

interface Electric {
    default void chargeBattery() { System.out.println("Charging battery..."); }
}

class Car extends Vehicle {
    void carType() { System.out.println("I am a Car"); }
}

// ElectricCar: single inheritance (extends Car) + interface (implements Electric)
// → Hybrid inheritance
class ElectricCar extends Car implements Electric {
    void display() {
        vehicleType();    // from Vehicle (2 levels up)
        carType();        // from Car
        chargeBattery();  // from Electric interface
        System.out.println("I am an Electric Car");
    }
}

class Main {
    public static void main(String[] args) {
        new ElectricCar().display();
    }
}
```

### 3. The super Keyword

super gives a subclass access to its direct parent's constructors and methods. It has two uses:

- **super(...)** — call a parent constructor. Must be the **first statement** in the subclass constructor. If omitted, Java inserts an implicit super() call to the no-arg parent constructor (which causes a compile error if no such constructor exists).
- **super.methodName()** — call the parent's version of an overridden method, useful when you want to extend (not replace) the parent's behaviour.

```
class Employee {
    String name;
    double baseSalary;

    Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    double calcSalary() {
        return baseSalary;
    }

    void display() {
        System.out.printf("Employee: %s Salary: %.2f%n", name, calcSalary());
    }
}

class Manager extends Employee {
    double bonus;

    Manager(String name, double baseSalary, double bonus) {
        super(name, baseSalary); // calls Employee(String, double)
        this.bonus = bonus;
    }

    @Override
    double calcSalary() {
        return super.calcSalary() + bonus; // extends parent's logic
    }

    @Override
    void display() {
        super.display(); // prints base info
        System.out.println(" (includes bonus: " + bonus + ")");
    }
}

class Main {
    public static void main(String[] args) {
        Manager m = new Manager("Abhilash", 50000, 10000);
        m.display();
        // Employee: Abhilash Salary: 60000.00
        // (includes bonus: 10000.0)
    }
}
```

**Constructor chaining order:**

When you create an object of a subclass, Java always builds the inheritance chain from the top down — the most senior ancestor's constructor runs first, then down to the subclass. This is why you always see parent output before child output in multilevel examples.

## 4. What Can a Subclass Do with Inherited Members?

| Action                        | How                                     | Notes                               |
|-------------------------------|-----------------------------------------|-------------------------------------|
| Use inherited field directly  | Just reference it (if public/protected) | Private fields need getters         |
| Declare new field (same name) | Field hiding — avoid; confusing         | Not recommended                     |
| Declare new field (new name)  | Normal new field                        | Common and clean                    |
| Use inherited method directly | Just call it                            | Runs parent's version               |
| Override instance method      | @Override + same signature              | Enables runtime polymorphism        |
| Hide static method            | Same signature, static in child         | Not overriding — compile-time only  |
| Declare new method            | Normal method def in subclass           | Only accessible via child reference |
| Call parent constructor       | super(...) as first statement           | Mandatory if no default parent ctor |
| Call parent method            | super.methodName()                      | From inside an override             |

## 5. Abstract Classes and Interfaces

Both provide abstraction in OOP. Knowing when to use each is a very common exam question and interview question.

| Feature              | Abstract Class                             | Interface                                            |
|----------------------|--------------------------------------------|------------------------------------------------------|
| Keyword              | abstract class                             | interface                                            |
| Instantiation        | Cannot instantiate directly                | Cannot instantiate directly                          |
| Methods              | Can have abstract + concrete methods       | All abstract by default; default/static since Java 8 |
| Fields               | Can have instance fields (any access)      | Only public static final (constants)                 |
| Constructor          | Can have constructors                      | No constructors                                      |
| Multiple inheritance | A class can extend only ONE abstract class | A class can implement MANY interfaces                |
| Use when             | Sharing code among closely related classes | Defining a contract for unrelated classes            |
| IS-A vs CAN-DO       | Models an IS-A relationship                | Models a CAN-DO (capability) relationship            |

```
// Abstract class – partial implementation
abstract class Shape {
    String color;
    Shape(String color) { this.color = color; }

    abstract double area();          // subclass MUST implement
    abstract double perimeter();    // subclass MUST implement

    void display() {                // concrete – shared by all subclasses
        System.out.printf("%s: area=%.2f perimeter=%.2f\n",
            color + " " + getClass().getSimpleName(), area(), perimeter());
    }
}

// Interface – capability contract
interface Drawable {
    void draw(); // any class that "can be drawn" implements this
}

interface Resizable {
    void resize(double factor);
}

class Circle extends Shape implements Drawable, Resizable {
    double radius;

    Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override public double area()      { return Math.PI * radius * radius; }
    @Override public double perimeter() { return 2 * Math.PI * radius; }
    @Override public void draw()        { System.out.println("Drawing circle r=" + radius); }
    @Override public void resize(double f) { radius *= f; }
}
```

```

class Main {
    public static void main(String[] args) {
        Circle c = new Circle("Red", 5);
        c.display();          // from abstract Shape
        c.draw();             // from Drawable
        c.resize(2.0);
        c.display();          // radius doubled
    }
}

```

## 6. The final Keyword in Inheritance

**final** is a modifier that **prevents extension or modification**. Exam papers frequently ask about it in the context of inheritance:

| Usage                  | Effect                                          | Example                                       |
|------------------------|-------------------------------------------------|-----------------------------------------------|
| final class            | Class cannot be subclassed                      | final class String — nobody can extend String |
| final method           | Method cannot be overridden in any subclass     | final void display() { ... }                  |
| final field / variable | Value cannot be reassigned after initialisation | final double PI = 3.14159;                    |

```

final class ImmutablePoint {
    final double x, y;
    ImmutablePoint(double x, double y) { this.x = x; this.y = y; }
    // x = 5; // COMPILE ERROR – final field
}
// class ColorPoint extends ImmutablePoint { } // COMPILE ERROR – final class

class Parent {
    final void greet() { System.out.println("Hello from Parent"); }
}
class Child extends Parent {
    // @Override void greet() { } // COMPILE ERROR – cannot override final method
}

```

## 7. The Object Class — Root of All Java Classes

Every class in Java implicitly extends `java.lang.Object` if no other superclass is specified. `Object` defines methods that every Java object inherits and can override:

| Method                        | Default behaviour                   | Override to...                                     |
|-------------------------------|-------------------------------------|----------------------------------------------------|
| <code>toString()</code>       | <code>ClassName@hashCode</code>     | Return a readable string of the object's state     |
| <code>equals(Object o)</code> | Checks identity ( <code>==</code> ) | Compare objects by content/value                   |
| <code>hashCode()</code>       | Memory-based integer                | Return value consistent with <code>equals()</code> |
| <code>getClass()</code>       | Returns runtime Class object        | Not overridable (final)                            |
| <code>clone()</code>          | Shallow copy                        | Deep copy of the object                            |

```

class Student {
    String name;
    int rollNo;

    Student(String name, int rollNo) {

```

```

        this.name    = name;
        this.rollNo = rollNo;
    }

    @Override
    public String toString() {
        return "Student{name=" + name + ", roll=" + rollNo + "}";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        Student other = (Student) o;
        return rollNo == other.rollNo && name.equals(other.name);
    }

    @Override
    public int hashCode() {
        return 31 * rollNo + name.hashCode();
    }
}

class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Abhilash", 101);
        Student s2 = new Student("Abhilash", 101);
        System.out.println(s1);           // Student{name=Abhilash, roll=101}
        System.out.println(s1 == s2);     // false (identity)
        System.out.println(s1.equals(s2)); // true (content equality)
    }
}

```

## 8. Design Patterns — Introduction & Classification

A **design pattern** is a general, reusable solution to a commonly occurring problem in software design. Patterns are not finished code — they are **templates / blueprints** that describe how to solve a problem in a particular context. They represent best practices accumulated by experienced developers.

The classic reference is the **Gang of Four (GoF)** book — "Design Patterns: Elements of Reusable Object-Oriented Software" (Gamma, Helm, Johnson, Vlissides, 1994). It defines 23 patterns organised into 3 categories:

### 8.1 Creational Patterns

Deal with **object creation**. Abstract the instantiation process so the system is independent of how objects are created, composed, and represented.

| Pattern          | Intent                                                                                             |
|------------------|----------------------------------------------------------------------------------------------------|
| Singleton        | Ensures only one instance of a class exists; provides a global access point                        |
| Factory Method   | Defines an interface for creating an object, but lets subclasses decide which class to instantiate |
| Abstract Factory | Provides an interface for creating families of related objects without specifying concrete classes |
| Builder          | Separates the construction of a complex object from its representation                             |
| Prototype        | Creates new objects by copying (cloning) an existing object                                        |

#### Singleton — code example (most common exam implementation):

```
class Singleton {
    // Only one instance ever – stored as a static field
    private static Singleton instance = null;

    // Private constructor – no one outside can call new Singleton()
    private Singleton() {
        System.out.println("Singleton instance created");
    }

    // Global access point
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    public void showMessage() {
        System.out.println("Hello from Singleton!");
    }
}

class Main {
    public static void main(String[] args) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();
        s1.showMessage();
        System.out.println(s1 == s2); // true – same instance
    }
}
```

#### Factory Method — code example:

```

// Product interface
interface Notification {
    void send(String message);
}

// Concrete products
class EmailNotification implements Notification {
    public void send(String msg) { System.out.println("Email: " + msg); }
}

class SMSNotification implements Notification {
    public void send(String msg) { System.out.println("SMS: " + msg); }
}

class PushNotification implements Notification {
    public void send(String msg) { System.out.println("Push: " + msg); }
}

// Factory – decides which product to create
class NotificationFactory {
    public static Notification create(String type) {
        switch (type.toLowerCase()) {
            case "email": return new EmailNotification();
            case "sms":   return new SMSNotification();
            case "push":  return new PushNotification();
            default: throw new IllegalArgumentException("Unknown: " + type);
        }
    }
}

class Main {
    public static void main(String[] args) {
        Notification n = NotificationFactory.create("email");
        n.send("Your OTP is 1234"); // Email: Your OTP is 1234
    }
}

```

## 8.2 Structural Patterns

Deal with **class and object composition**. Define how to assemble objects and classes into larger structures while keeping those structures flexible and efficient.

| Pattern   | Intent                                                                                                                 |
|-----------|------------------------------------------------------------------------------------------------------------------------|
| Adapter   | Converts the interface of a class into another interface that clients expect; makes incompatible interfaces compatible |
| Bridge    | Decouples an abstraction from its implementation so the two can vary independently                                     |
| Composite | Composes objects into tree structures to represent part-whole hierarchies                                              |
| Decorator | Attaches additional responsibilities to an object dynamically                                                          |
| Facade    | Provides a simplified interface to a complex subsystem                                                                 |
| Flyweight | Uses sharing to support large numbers of fine-grained objects efficiently                                              |
| Proxy     | Provides a surrogate or placeholder for another object to control access to it                                         |

## 8.3 Behavioral Patterns

Deal with **communication and responsibility between objects**. Characterise how classes and objects interact and distribute responsibility.

| Pattern                 | Intent                                                                                         |
|-------------------------|------------------------------------------------------------------------------------------------|
| Iterator                | Provides a way to access elements of a collection sequentially without exposing the underlying |
| Observer                | Defines a one-to-many dependency so that when one object changes state, all its dependents     |
| Strategy                | Defines a family of algorithms, encapsulates each one, and makes them interchangeable          |
| Command                 | Encapsulates a request as an object, allowing parameterisation and queuing of requests         |
| Template Method         | Defines the skeleton of an algorithm in a base class, deferring some steps to subclasses       |
| State                   | Allows an object to alter its behaviour when its internal state changes                        |
| Chain of Responsibility | Passes a request along a chain of handlers until one handles it                                |
| Mediator                | Defines an object that encapsulates how a set of objects interact                              |
| Memento                 | Captures and externalises an object's internal state for later restore                         |
| Visitor                 | Lets you add further operations to objects without having to modify them                       |
| Interpreter             | Defines a grammar for a language and an interpreter for the grammar                            |

## 9. The Iterator Pattern (Detailed)

The Iterator is a **Behavioral design pattern** that provides a standard way to traverse (loop through) the elements of a collection — whether it is an array, a linked list, a tree, or any other structure — **without exposing its internal implementation**. The client code that iterates doesn't need to know how the collection is built internally.

### Key components:

| Component            | Role                                                                         |
|----------------------|------------------------------------------------------------------------------|
| Iterator interface   | Declares hasNext() and next() (and optionally remove())                      |
| Concreteliterator    | Implements Iterator; tracks the current position in the collection           |
| Aggregate / Iterable | The collection; declares createliterator() or iterator()                     |
| ConcreteAggregate    | Implements the collection; returns a Concreteliterator                       |
| Client               | Uses only the Iterator interface — doesn't know about the internal structure |

### 9.1 Using Java's Built-in Iterator (java.util)

All Java collections (ArrayList, LinkedList, HashSet, etc.) already implement Iterable<T> which provides an Iterator<T>. This is the iterator pattern baked into the JDK.

```
import java.util.ArrayList;
import java.util.Iterator;

class Main {
    public static void main(String[] args) {
        ArrayList<String> subjects = new ArrayList<>();
        subjects.add("OOP");
        subjects.add("DBMS");
        subjects.add("OS");
        subjects.add("CN");

        // Method 1: explicit Iterator
        Iterator<String> it = subjects.iterator();
        while (it.hasNext()) {
            String s = it.next();
            System.out.println(s);
            if (s.equals("OS")) it.remove(); // safe removal during iteration
        }

        // Method 2: for-each loop (internally uses Iterator)
        for (String s : subjects) {
            System.out.println(s);
        }

        // Method 3: forEach + lambda (Java 8+)
        subjects.forEach(s -> System.out.println(s));
    }
}
```

### 9.2 Implementing Iterator Pattern from Scratch

This is the version examiners expect — implementing the pattern manually to demonstrate you understand it, not just using ArrayList.

```

// Step 1: Iterator interface
interface Iterator<T> {
    boolean hasNext();
    T next();
}

// Step 2: Aggregate interface
interface Container<T> {
    Iterator<T> getIterator();
}

// Step 3: Concrete collection – a simple name repository
class NameRepository implements Container<String> {
    private String[] names;
    private int size;

    NameRepository(int capacity) {
        names = new String[capacity];
        size = 0;
    }

    void add(String name) {
        if (size < names.length) names[size++] = name;
    }

    // Step 4: ConcreteIterator – defined as inner class
    @Override
    public Iterator<String> getIterator() {
        return new Iterator<String>() {
            int index = 0;

            @Override
            public boolean hasNext() {
                return index < size;
            }

            @Override
            public String next() {
                return names[index++];
            }
        };
    }
}

// Step 5: Client
class Main {
    public static void main(String[] args) {
        NameRepository repo = new NameRepository(5);
        repo.add("Abhilash");
        repo.add("Riya");
        repo.add("Arjun");

        Iterator<String> it = repo.getIterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
        // Output:
        // Abhilash
        // Riya
        // Arjun
    }
}

```

```
}
```

### Why Iterator Pattern matters:

| Benefit               | Explanation                                                            |
|-----------------------|------------------------------------------------------------------------|
| Single Responsibility | Separates traversal logic from the collection itself                   |
| Open/Closed Principle | Add new iterators (reverse, filtered) without modifying the collection |
| Uniform interface     | All collections traversed the same way — client code is generic        |
| Safe removal          | iterator.remove() is the only safe way to remove during traversal      |
| Hides implementation  | Client doesn't know if collection is an array, linked list, or tree    |

## 10. Inheritance vs. Composition

A classic design question: when should you use inheritance, and when should you use composition (one class **has-a** reference to another)?

| Aspect       | Inheritance (IS-A)                           | Composition (HAS-A)                            |
|--------------|----------------------------------------------|------------------------------------------------|
| Relationship | Dog IS-A Animal                              | Car HAS-A Engine                               |
| Coupling     | Tight — child depends on parent internals    | Loose — connected via interface/reference      |
| Reuse        | Reuses code but tied to parent hierarchy     | Reuses via delegation; easy to swap components |
| Flexibility  | Rigid — changing parent breaks children      | Flexible — swap the component at runtime       |
| Java         | extends                                      | Field reference + delegation                   |
| Prefer when  | Genuine IS-A relationship; need polymorphism | HAS-A; or when relationship may change         |

**Rule of thumb:** Prefer composition over inheritance unless there is a genuine, stable IS-A relationship. Inheritance is powerful but creates tight coupling — deep hierarchies (4+ levels) are a code smell.

## 11. Likely Semester-Exam Questions

### Q1. Define inheritance. What are its advantages and disadvantages?

Inheritance is the mechanism by which a subclass acquires the fields and methods of a superclass, enabling code reuse and IS-A modelling. Advantages: code reusability, method overriding enabling polymorphism, class hierarchy for real-world modelling, easier code maintenance. Disadvantages: tight coupling (parent changes break children), complexity in deep hierarchies, can violate encapsulation if parent internals are exposed.

### Q2. What are the types of inheritance in Java? Which type does Java NOT support with classes and why?

Java supports single, multilevel, hierarchical, multiple (via interfaces only), and hybrid (via interfaces only). Multiple inheritance of classes is not supported to avoid the Diamond Problem — ambiguity when two parent classes define the same method and a child inherits both without knowing which version to use. Interfaces solve this since default method conflicts must be explicitly resolved.

### Q3. What is the role of the super keyword?

super has two uses: (1) super(...) invokes the parent class constructor and must be the first statement in the subclass constructor — used to initialise inherited state. (2) super.methodName() calls the parent class's overridden method — used to extend, not replace, the parent's behaviour.

### Q4. What is the difference between abstract class and interface?

Abstract class: can have concrete + abstract methods, instance fields, constructors, any access modifier — a class extends only ONE. Interface: all abstract by default (default/static methods allowed since Java 8), only public static final fields, no constructors — a class can implement MANY. Use abstract class for IS-A with shared code; use interface for CAN-DO capability contracts across unrelated classes.

### Q5. What does the final keyword do in the context of inheritance?

final class: cannot be subclassed (e.g. String). final method: cannot be overridden in any subclass. final field/variable: value cannot be reassigned after initialisation. All three prevent extension/modification to lock down behaviour or value.

### Q6. What are design patterns? How are they classified?

Design patterns are reusable solutions to commonly occurring software design problems. Classified into 3 GoF categories: (1) Creational — object creation (Singleton, Factory, Builder, Prototype, Abstract Factory). (2) Structural — class/object composition (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy). (3) Behavioral — object communication/responsibility (Iterator, Observer, Strategy, Command, Template Method, State, Chain of Responsibility, etc.).

### Q7. Explain the Iterator design pattern with a Java example.

Iterator is a behavioral pattern that provides sequential access to elements of a collection without exposing its internal structure. Key components: Iterator interface (hasNext(), next()), ConcreteIterator (tracks position), Aggregate/Iterable (provides getIterator()), ConcreteAggregate (the actual collection). Java's java.util.Iterator is a built-in implementation; every Collection implements Iterable. See Section 9 code.

### Q8. What is the IS-A relationship? How is it tested in Java?

IS-A is the inheritance relationship — a Dog IS-A Animal because Dog extends Animal. In Java it is tested with the instanceof operator: (dog instanceof Animal) returns true. All classes also have an IS-A relationship with Object since every class ultimately extends Object implicitly.

### **Q9. What is the Diamond Problem? How does Java avoid it?**

The Diamond Problem occurs when class D inherits from both B and C, and both B and C inherit from A and override the same method — it is ambiguous which version D should inherit. Java avoids it by prohibiting multiple class inheritance. With interfaces that provide default methods, if two interfaces conflict, the implementing class must explicitly override the method to resolve the ambiguity.

---

End of Unit 3 notes. Sources: GFG Inheritance in Java (types, IS-A, advantages/disadvantages), Oracle Java Tutorials (subclasses, what a subclass can do, casting, Object class), Code World / arnab7070 (design patterns classification, Iterator Pattern), Medium / Gaurav Sahu (Diamond Problem, composition vs inheritance). Abstract class vs interface, final keyword, Singleton/Factory/Iterator implementations, and exam Q&A written for this syllabus.